

1. An outline of the reconstruction algorithm in high-level pseudo-code.

- a. Create a 0-image (B_R) with the same size as the skeleton image (M)
- b. Raster-Scan M :
If pixel $M(i, j) > 0$ then assign 255 to any pixels in the range of $B_R(i-M(i, j), j-M(i, j))$ to $B_R(i+M(i, j), j+M(i, j))$

2. A well-documented hardcopy of the source code for the computation of the Medial Axis and the reconstruction of the original binary image from the Medial Axis.

Computer Vision HW#2

Nima Darbandi
811296007

nimadarbandi / ComputerVision

Type to search

<> Code

Issues

Pull requests

Actions

Projects

Wiki

Security

Insights

Settings

ComputerVision / Skeletonization / skeletonizer.py

...

nimadarbandi

updated chessboard

8646e59 · now

History

ComputerVision / Skeletonization / skeletonizer.py

↑ Top

Code

Blame

117 lines (91 loc) · 4.06 KB

Raw

```
15
16
17 #####Functions#####
18 #####
19
20 def iterative_distance_propagation(binary_img):
21
22     rows, cols = binary_img.shape
23
24     # Corrected input image: Foreground = inf, Background = 0
25     distance_image = np.where(binary_img == 0, 0, np.inf)
26
27     iteration = 0
28     while True:
29         iteration += 1
30         changed = False
31         temp_img = distance_image.copy()
32
33         for i in range(0, rows):
34             for j in range(0, cols):
35                 if temp_img[i, j] > 0: # foreground pixels
36                     neighbors = neighbors(temp_img, i, j)
37                     min_value = min(neighbors) + 1
38                     if min_value < temp_img[i, j]:
39                         temp_img[i, j] = min_value
40                         changed = True
41
42         distance_image = temp_img.copy()
43
44         # Convergence
45         if not changed:
46             break
47
48         print(f"IDP converged after {iteration} iterations.")
49         return distance_image
50
51
52
53
54 def neighbors(dist_img, i, j):
55     neighbors = []
56     rows, cols = dist_img.shape
57     for di in [-1, 0, 1]:
58         for dj in [-1, 0, 1]:
59             if di == 0 and dj == 0:
60                 continue # center pixel
61             ni = i + di
```

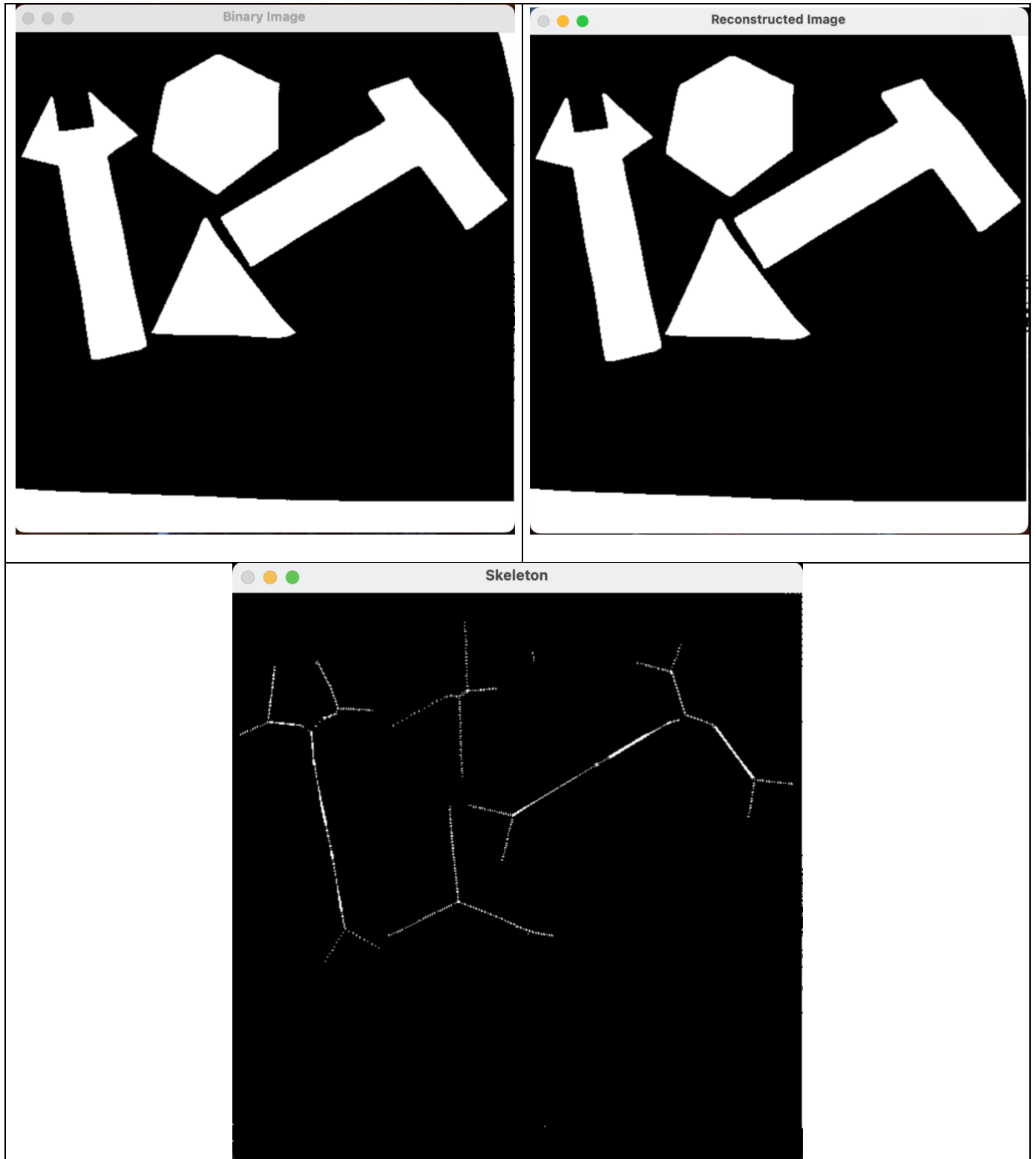
Computer Vision HW#2

Nima Darbandi

811296007

```
62         nj = j + dj
63         if 0 <= ni < rows and 0 <= nj < cols:
64             neighbors.append(dist_img[ni, nj])
65     return neighbors
66
67
68     # Extract the skeleton from the distance map
69     def extract_skeleton(distance_image):
70         rows, cols = distance_image.shape
71         skeleton = np.zeros_like(distance_image, dtype=np.uint8)
72
73         for i in range(0, rows):
74             for j in range(0, cols):
75                 if distance_image[i, j] > 0:
76                     # Local maximum in distance map
77                     if distance_image[i, j] >= max(neighbors(distance_image, i, j)):
78                         skeleton[i, j] = distance_image[i, j]
79     return skeleton
80
81     def reconstruct_image(skeleton):
82         rows, cols = skeleton.shape
83         image_M = np.zeros((rows, cols), dtype=np.uint8)
84         for i in range(0, rows):
85             for j in range(0, cols):
86                 if skeleton[i, j] > 0:
87                     radius = int(skeleton[i, j])
88                     for m in range(i-radius, i+radius):
89                         for n in range(j-radius, j+radius):
90                             if 0 <= m < rows and 0 <= n < cols:
91                                 image_M[m, n] = 255
92     return image_M
93
94     #####
95     # Main
96     #####
97
98     binary_image = np.fromfile(input_file, dtype=np.uint8)
99     binary_image = binary_image.reshape((512, 512)) # adjust dimensions as needed
100    if foreground == 0: # inverse the images with white background
101        binary_image = 255 - binary_image
102
103    distance_map = iterative_distance_propagation(binary_image)
104
105
106    skeleton_M = extract_skeleton(distance_map)
107    skeleton_image = np.where(skeleton_M > 0, 255, 0).astype(np.uint8)
108    recons_image = reconstruct_image(skeleton_M)
109
110    cv2.imshow('Binary Image', binary_image)
111    cv2.imshow('Skeleton', skeleton_image)
112    cv2.imshow('Reconstructed Image', recons_image)
113    cv2.waitKey(0)
114    cv2.destroyAllWindows()
115    skeleton_M.tofile(skeleton_file)
116    recons_image.tofile(reconstructed_file)
117    distance_map.tofile(distance_file)
```

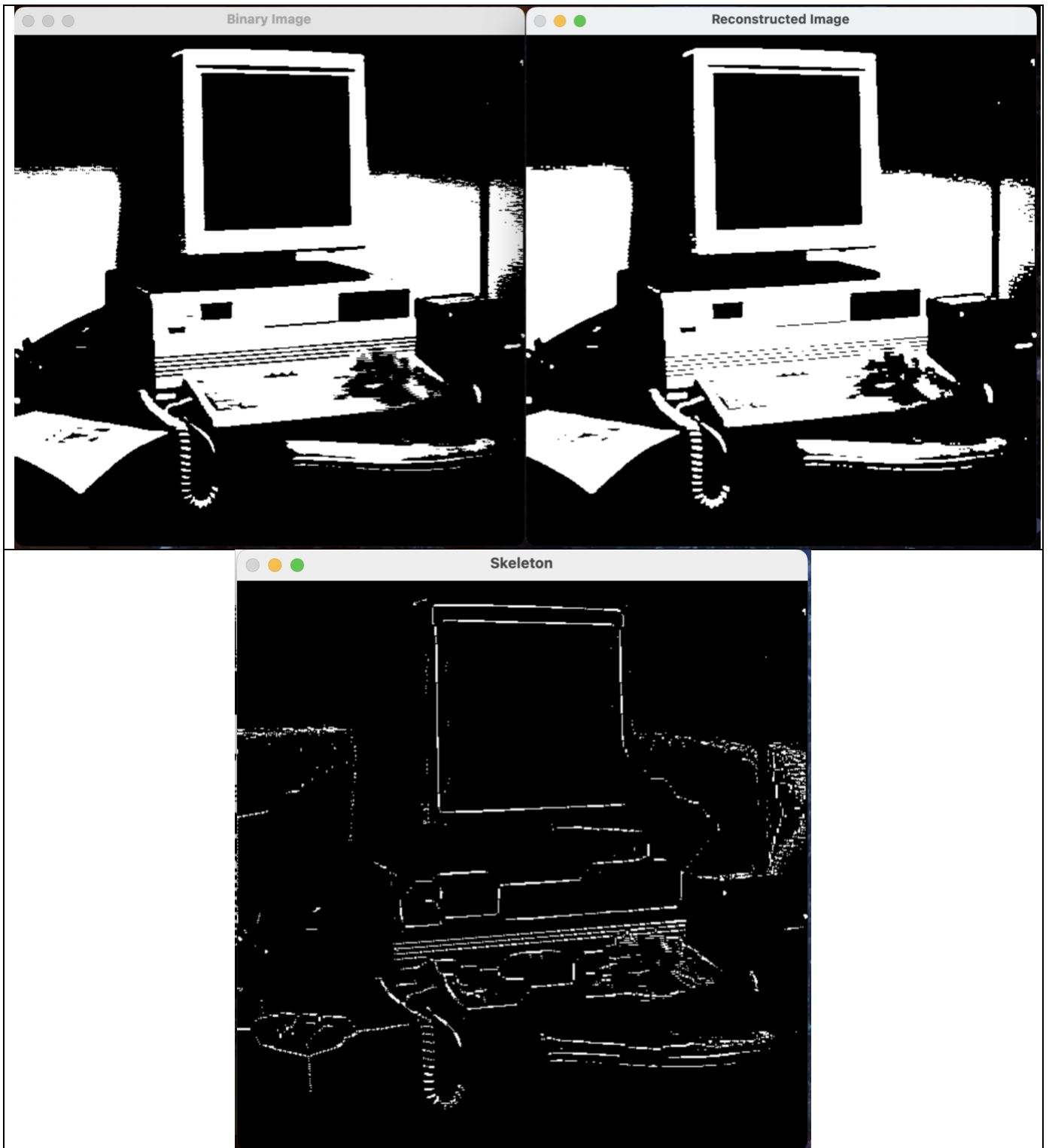
3. Hardcopies of the images B, M, and B_R.



Computer Vision HW#2

Nima Darbandi

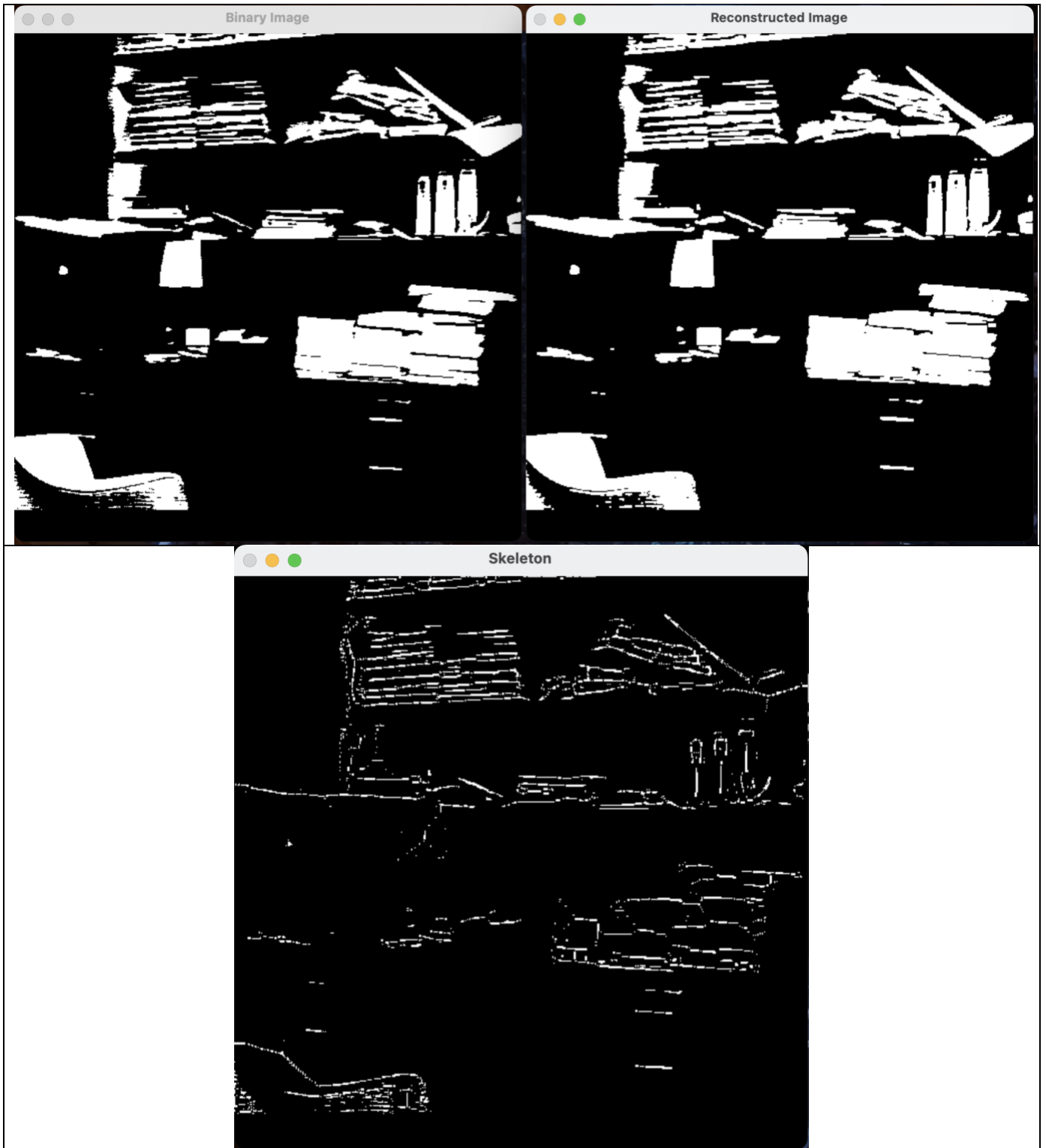
811296007



Computer Vision HW#2

Nima Darbandi

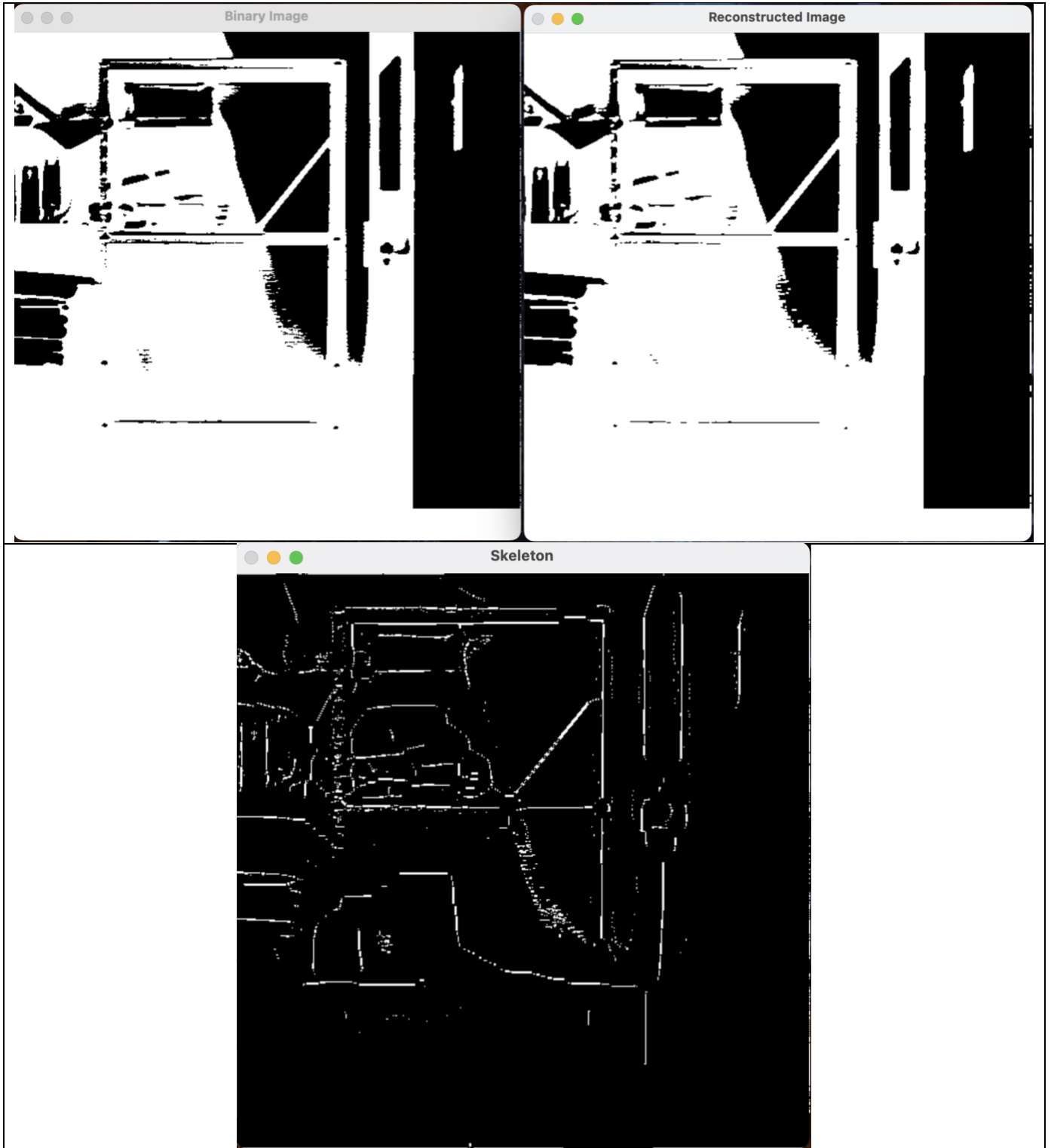
811296007



Computer Vision HW#2

Nima Darbandi

811296007



4. Comments on the results obtained.

a. The impact of different distance types:

I tried Manhattan and Chessboard distancing, and my experience showed that using Manhattan distancing algorithm for creating the distance image is not sufficiently accurate to reconstruct the image from the skeleton file, and the reconstructed image showed major loss during construction. However, chessboard distancing was enough, and the initial binary image and the reconstructed ones showed almost the same details.

b. About the background of the images

The binary images comb and test* had different color codes for the background. I changed everyone of them to 0 for background and 255 for foreground to draw the skeleton on white pixels.